Online Resource 1

Supplementary Material for "Decoding Delusions: An Empirical Study on LLM Code Hallucination"

Prompts, Annotation Guidelines, Adjudication Cases, and Supplementary Experiments

This document accompanies the paper. It contains the complete generation and type-classification prompts (Section S1), the annotation guidelines and reuse protocol (Section S2), adjudication cases for confusable category boundaries (Section S3), the temperature-control experiment (Section S4), and the type-classification evaluation rules and per-category results (Section S5).

S1 Generation and Classification Prompts

S1.1 Generation Prompts

Generation uses a shared system message and one of two user templates. HumanEval tasks use the function/method-completion template; APPS tasks use it when starter code is supplied and the standalone-script template otherwise. The problem description is taken from the `question` field and the signature from the function or method definition in `starter_code`.

Shared system message:

```
You are an expert programmer focused on writing clean, efficient, and correct Python code to
  solve the given problem. Your response should be a complete, runnable code solution that
  strictly adheres to the task requirements.
```

User message for function/method completion:

```
Task: Complete the Python code for the provided function/method signature to solve the
  problem.

Problem Description:
<Problem description from the 'question' field>

Code Signature to Implement:
<The function/method definition line from 'starter_code'>

Instructions:
- Your response must contain the full, complete implementation of the provided class or
  function.
- Do not add any code for reading input or printing output.
- Do not include test cases or example usage.
```

User message for standalone script generation:

```
Task: Write a complete, standalone Python script to solve the algorithmic problem.

Problem Description:
<Problem description from the 'question' field>

Execution Requirements:
- The script must read all required data from standard input (stdin).
- The script must write the final answer to standard output (stdout).
```

```
- The entire solution, including input/output handling, must be contained in a single script.

Instructions:
- Analyze the problem to determine the correct input format and output format.
- Write a complete script that can be executed directly to solve the problem.
```

S1.2 Type Classification Prompt

All five LLMs use the following prompt. {{TASK_SPEC}} is replaced by the task statement and interface, and {{NUMBERED_CODE}} by the model output with line numbers. Each example is a minimal single-issue case derived from the annotation guidelines (Section S2) and does not overlap with the test samples. The following blocks form a single prompt and are concatenated in the order shown.

```
[SYSTEM]
You are an expert Python code reviewer. You classify issues in LLM-generated Python code
  according to the CoDelusion taxonomy of code hallucinations.

[USER]
The code below was generated by an LLM for the given task. Classify its issue into exactly
  one of the thirteen subcategories defined below. Judge the submitted code against the task
  specification, the code context, Python and API specifications, and explicit execution
  constraints. Inputs excluded by the task and defensive checks that the task does not
  require are not grounds for a label, and a different but valid solution is not an issue.
  Each definition is followed by one example.
```

```
1.1 DTM (Data Type Mismatch): An operation, argument, or return value uses a type or value
  that violates its contract.
Example:
Task: Implement age_text(age: int) -> str. Return the text "Age: " followed by the decimal
  representation of age. Age is an integer from 0 to 150.
Code:
1 | def age_text(age):
2 |     return "Age: " + age
Answer: {"label": "DTM", "lines": [2], "reason": "The + operation combines a str with an int;
  age=8 requires \"Age: 8\", but the code raises TypeError."}
```

```
1.2 DSM (Data Structure Misinterpretation): The code misinterprets the actual form of a data
  structure given by the task or built by the code, for example by accessing a nonexistent
  key, index, or field. Members of external libraries belong to AKE.
Example:
Task: Implement get_name(record). record is a dict with exactly the keys "name" and "age".
  Return the value of "name".
Code:
1 | def get_name(record):
2 |     return record["username"]
Answer: {"label": "DSM", "lines": [2], "reason": "The supplied dictionary schema contains no
  \"username\" key; the code accesses a nonexistent field."}
```

```
2.1 GLM (Global Logic Misalignment): The main computational strategy cannot accomplish the
  task: the code computes a different objective or relies on an assumption that does not hold
  for the task. The number of lines involved does not matter.
Example:
Task: Implement max_subarray_sum(xs). xs is a nonempty list of integers. Return the maximum
  sum of a nonempty contiguous subarray.
Code:
1 | def max_subarray_sum(xs):
2 |     return sum(xs)
Answer: {"label": "GLM", "lines": [2], "reason": "Summing the entire list is not a
  maximum-subarray strategy. For [-2, 3, -1], it returns 0 instead of 3."}
```

```
2.2 CI (Context Inconsistency): The code contradicts explicit information in the task or the
  given code, or contradicts code the model itself has written, such as an undefined variable
  or a call inconsistent with its own definition.
Example:
Task: Implement same_identifier(a, b) for two strings. Identifiers are explicitly
  case-sensitive: "A" and "a" are different identifiers.
Code:
1 | def same_identifier(a, b):
2 |     return a.lower() == b.lower()
Answer: {"label": "CI", "lines": [2], "reason": "Case normalization contradicts the explicit
  case-sensitive identifier contract; \"A\" and \"a\" are incorrectly treated as equal."}
```

```
2.3 LoLM (Local Logic Misalignment): The main strategy is appropriate, but a local condition,
  comparison, index, loop bound, state update, or initialization is wrong.
Example:
Task: Implement factorial(n) for integer 0 <= n <= 20. Return n!.
Code:
1 | def factorial(n):
2 |     result = 1
3 |     for i in range(1, n):
4 |         result *= i
5 |     return result
Answer: {"label": "LoLM", "lines": [3], "reason": "The multiplication strategy is
  appropriate, but range(1, n) omits n. For n=4, the result is 6 rather than 24."}
```

```
2.4 II (Incomplete Implementation): A required functionality, branch, return value, or
  delivery component is missing or replaced by a placeholder such as pass or TODO. In
  standalone-script tasks, missing stdin/stdout handling is also II; function-completion
  tasks do not require input/output code.
Example:
Task: Implement summary(xs) for a nonempty integer list. Return a tuple (sum of the elements,
  maximum element).
Code:
1 | def summary(xs):
2 |     total = sum(xs)
3 |     # Maximum-element computation and the required tuple are not implemented.
4 |     return total
Answer: {"label": "II", "lines": [3, 4], "reason": "The maximum-element result and the
  required two-component return value are absent."}
```

```
3.1 CD (Code Duplication): An identifiable code fragment or computation is repeated
  unnecessarily. A duplicated fragment whose result is unused is CD, not DC.
Example:
Task: Implement total(xs) for a list of at most 100 integers. Return the sum of its elements.
Code:
1 | def total(xs):
2 |     result = 0
3 |     for x in xs:
4 |         result += x
5 |     result = 0
6 |     for x in xs:
7 |         result += x
8 |     return result
Answer: {"label": "CD", "lines": [2, 3, 4, 5, 6, 7], "reason": "The sum is computed twice by
  duplicated loops; the first computation is immediately discarded and the second repeats
  it."}
```

```
3.2 DC (Dead Code): Code is unreachable, or is executed but its result never affects the
  program state or output.
Example:
Task: Implement absolute(x) for an integer x. Return its absolute value.
```

```
Code:
1 | def absolute(x):
2 |     return abs(x)
3 |     unused = x * x
Answer: {"label": "DC", "lines": [3], "reason": "The assignment after the unconditional
  return is unreachable."}
```

```
4.1 AKE (API Knowledge Errors): The code contradicts the facts of an external library,
  module, or API, such as calling a nonexistent function, using a wrong signature, misusing
  return semantics, or importing a nonexistent module.
Example:
Task: Implement is_prime(n) for an integer 2 <= n <= 100. Use only Python standard-library
  facilities or your own implementation.
Code:
1 | import math
2 | def is_prime(n):
3 |     return math.isprime(n)
Answer: {"label": "AKE", "lines": [3], "reason": "The standard-library math module does not
  provide an isprime function."}
```

```
5.1 RSI (Robustness & Security Issues): Under inputs admitted by the task or explicit
  operating conditions, an unhandled boundary case or exception causes a crash, or the code
  uses its inputs unsafely and contains a security vulnerability (for example, eval on
  untrusted input). Failures caused by excessive time, memory, or recursion demand belong to
  RO.
Example:
Task: Implement evaluate(expression) for untrusted user input. Accept only integer literals,
  +, -, *, and parentheses. Reject other syntax and never execute arbitrary Python code.
Code:
1 | def evaluate(expression):
2 |     return eval(expression)
Answer: {"label": "RSI", "lines": [2], "reason": "Unvalidated user input is passed to eval,
  which permits arbitrary Python expression execution rather than the restricted arithmetic
  grammar."}
```

```
6.1 RO (Resource Overuse): Under the input scale and resource limits given by the task, the
  time, memory, recursion depth, or numeric range required by the algorithm exceeds the
  limit, including infinite loops.
Example:
Task: Implement count_units(n), returning n for integer 0 <= n <= 100000000. Additional
  working memory must remain below 64 MiB.
Code:
1 | def count_units(n):
2 |     units = b"x" * n
3 |     return len(units)
Answer: {"label": "RO", "lines": [2], "reason": "For n=100000000, the bytes payload alone is
  100000000 bytes, exceeding the explicit 64 MiB working-memory bound."}
```

```
7.1 SE (Syntax Errors): The code violates Python syntax and cannot be parsed.
Example:
Task: Implement double(x), returning 2*x for an integer x.
Code:
1 | def double(x)
2 |     return 2 * x
Answer: {"label": "SE", "lines": [1], "reason": "The function definition is missing its
  required colon and cannot be parsed as Python."}
```

```
7.2 NCC (Non-Code Content): The response supplies prose, pseudocode, or placeholder text
  instead of the requested executable code, or mixes natural language into code lines.
Example:
Task: Implement total(xs) as executable Python code, returning the sum of a list of integers.
Code:
```

```
1 | Add all the elements with a loop and then return the accumulated value.
Answer: {"label": "NCC", "lines": [1], "reason": "The response describes an approach in prose
  but supplies no executable implementation."}
```

```
Output format:
Return exactly one JSON object and no other text:
{"label": "<LABEL>", "lines": [<line numbers>], "reason": "<brief, checkable evidence>"}
<LABEL> must be exactly one of DTM, DSM, GLM, CI, LoLM, II, CD, DC, AKE, RSI, RO, SE, NCC. If
  the code contains several issues, return the label of the most evident one. If none of the
  listed issues is supported, return {"label": "NONE", "lines": [], "reason": "<brief
  explanation>"}. Line numbers refer to the numbered code below.

Task:
{{TASK_SPEC}}

Code:
{{NUMBERED_CODE}}
```

S2 Annotation Guidelines and Reuse Protocol

Rule numbers (Rule x.y) follow the guideline released in the repository (v1.0); adjudication records cite them.

S2.1 Annotation Object and Principles

An annotation unit comprises the task specification (problem statement, interface, explicit constraints), the model's raw output, the generation mode (function/method completion or standalone script; Rule 6.1), and the test-execution results. Labels describe the raw output; code produced by I/O adaptation, code extraction, or annotator repair is auxiliary evidence only.

- **Rule 2.1 (Evidence).** Each label corresponds to one specific issue and records the code location (line numbers), the violated requirement (task specification, context, language or API specification, or execution constraint), and the evidence (counterexample input, execution result, specification clause, or static reasoning). If any of the three is missing, no label is assigned.
- **Rule 2.2 (Independent issues).** Independent issues are labeled separately; the sample's label set is the union of their labels. A subcategory occurring several times is recorded once per sample.
- **Rule 2.3 (One defect, one label).** Consequences of a defect (exceptions, test failures, wrong output) receive no extra label: a call to a non-existent API that crashes the program is AKE only, without RSI or LoLM. A defect matching several subcategories receives the most specific one (Rule 5.2).
- **Rule 2.4 (Permitted inputs).** Only the input domain permitted by the problem and explicitly stated operating conditions count. Excluded inputs and defensive checks the problem does not require are no grounds for a label.
- **Rule 2.5 (Alternative solutions).** An implementation that differs from the reference solution but meets the task requirements is not labeled.
- **Rule 2.6 (Empty label set).** An empty set means that no listed issue was found under this guideline and the available evidence, not that the code is proven correct. Samples that pass all tests are still checked item by item, including for redundancy, dead code, and logic errors the tests miss.

S2.2 Evidence Sources and Priority

Evidence is admitted in the following order; an earlier source prevails over a conflicting later one:

1. task text and the explicitly given interface, input domain, and constraints;
2. code context in the prompt and code the model has already written in the same output;
3. Python language specification and library documentation of the relevant version;
4. reproducible execution evidence: benchmark test results, minimal counterexamples supplied by annotators, and records of resource-limit violations;
5. reference solution (for reference only).

If the problem statement, tests, and reference solution contradict one another, the sample is marked "specification under verification" and referred to the adjudicator (Rule 11.4); no label is assigned while the contradiction stands.

S2.3 Category Criteria

The taxonomy has seven top-level categories and thirteen subcategories. Data & Structure Issues, Logic Issues, and External Knowledge Issues correspond to input-conflicting, context-conflicting, and fact-conflicting hallucinations; studies of hallucination in the strict sense may use only these three categories. Redundancy & Dead Code, Robustness and Security Issues, Resource Management Issues, and Syntax & Specification Issues record redundancy, non-functional defects, and output-form anomalies. Tables S1 and S2 give the criteria for each subcategory (Rule 4.1).

S2.4 Decision Order and Boundary Rules

Rule 5.1 (Decision order). Each sample is checked in the order below; a finding at one step does not end the check for independent issues at later steps.

1. Output form: executable code (NCC); parsable (SE).
2. Delivery completeness: required functionality and interface present for the generation mode (II).
3. External knowledge and data: correct use of APIs and modules (AKE), of types (DTM), and of data structures (DSM).
4. Logic: main strategy can accomplish the task (GLM); explicit context respected (CI); local implementation correct (LoLM).
5. Non-functional properties: under permitted inputs and conditions, boundaries and exceptions handled and inputs used safely (RSI); resource demand within the limits (RO).
6. Redundancy: unnecessary duplication (CD); ineffective code (DC).

Boundary rules (Rules 5.2–5.8; worked cases in Section S3):

- **Rule 5.2 (Specificity first).** A defect that fits several subcategories receives the most specific one: a type-contract error is DTM, a data-structure error DSM, an external API error AKE, and a violation of explicit context CI. LoLM or GLM applies only when no more specific subcategory does.

Table S1: Decision criteria for Data & Structure Issues and Logic Issues (Rule 4.1)

Sub-category	**Include when**	**Exclude / assign elsewhere**	**Minimum evidence**
1 Data & Structure Issues			
DTM 1.1	The type or value of an operation, argument, or return value violates its contract (e.g., adding a string to an integer; passing a float where an integer is required)	Type conversions the task requires; type problems arising only under inputs the problem excludes	Operation, type contract, permitted triggering input
DSM 1.2	Access to an index, key, field, or member that mismatches the actual shape of the data structure (e.g., a missing dictionary key; a nested list treated as flat)	Members of external-library objects → AKE; off-by-one boundary index with the structure understood → LoLM	Actual shape, access statement, permitted input exposing the conflict
2 Logic Issues			
GLM 2.1	The main strategy cannot accomplish the task: it computes a different objective, or its core algorithm relies on an assumption that does not hold for the task; in extreme cases the intent is unrecognizable	Feasible main strategy with only local errors → LoLM; the number of wrong lines is no criterion	Required input–output relation, relation actually computed, counterexample separating them
CI 2.2	Contradiction of explicit context: a convention in the problem statement or given code (e.g., "stack", "case-sensitive"), or code the model already wrote in the same output (undefined or inconsistently spelled variable; call inconsistent with its own definition)	No explicit context violated, merely a computational error → LoLM; specific type, structure, or API errors → DTM, DSM, AKE	Violated context (text or code line) and location of the contradicting code
LoLM 2.3	The main strategy is applicable, but a condition, comparison, index, loop bound, state update, or initialization is wrong	Wrong overall objective → GLM; wholly missing functionality → II; explicit context violated → CI	Applicable algorithmic structure, erroneous local part, minimal counterexample
II 2.4	A required functionality, branch, return content, or deliverable component is missing, or a placeholder (`pass`, TODO) replaces it; in standalone-script mode, required I/O handling is missing	Missing I/O code in function/method completion mode (Rule 6.1); implemented but wrong → GLM or LoLM by the nature of the error	Required components matched to the missing parts

Table S2: Decision criteria for Redundancy & Dead Code, External Knowledge Issues, Robustness and Security Issues, Resource Management Issues, and Syntax & Specification Issues (Rule 4.1)

Sub-category	Include when	Exclude / assign elsewhere	Minimum evidence
3 Redundancy & Dead Code			
CD 3.1	Identifiable, unnecessary duplicated code or repeated computation, including verbatim copies of code already in the prompt	Similar structures needed for different branches or data; no extra DC for the same duplicated segment (Rule 5.4)	Locations of the duplicates; why the duplication has no independent effect
DC 3.2	Code unreachable on every execution path, or executed code whose result is never used and affects neither state nor output	Logging, debugging, or validation output the task requires; repeated computation → CD	Grounds for unreachability, or data flow showing a defined variable is never used
4 External Knowledge Issues			
AKE 4.1	Contradiction of facts about an external library, module, or API: non-existent function or method, wrong parameter signature, misused return-value semantics, import of a non-existent module, call to a module not imported	Keys or fields of user data structures → DSM; a genuinely existing dependency absent from the evaluation environment (not a model error)	Library and version, correct interface specification, conflicting call
5 Robustness and Security Issues			
RSI 5.1	Under permitted inputs or explicit operating conditions: a crash from an unhandled boundary case or exception (e.g., an empty input the task admits); a security vulnerability from unsafe use of inputs (e.g., `eval` on untrusted input)	Defensive checks the problem does not require; ordinary computational errors at a boundary (Rule 5.6); failures from excessive time, memory, or recursion demand, even when raised as exceptions → RO	Permitted trigger and failure path, or location of the security risk
6 Resource Management Issues			
RO 6.1	Under the task's input scale and resource limits, the algorithm's inherent demand for time, memory, recursion depth, or numeric range exceeds the limits, including infinite and unbounded loops; numeric overflow applies only to floating-point or fixed-width types (Python integers do not overflow)	High complexity within the limits; exceeding only the protective thresholds of the evaluation environment (Rule 6.4)	Input scale, applicable limit, and an execution record of the violation or a computable lower bound on demand
7 Syntax & Specification Issues			
SE 7.1	The code violates Python syntax and cannot be parsed	Parse failure caused by mixed-in non-code text → NCC (Rule 5.5); format corruption by the extraction script (not a model error)	Raw code fragment and parser error message
NCC 7.2	Explanatory text, pseudocode, comments only, or placeholder text in place of the required executable implementation, or natural language mixed into code lines	Outer Markdown code fences; normal comments; complete structure with only a `pass` placeholder → II	Raw output compared with the requirement to deliver executable code

- **Rule 5.3 (GLM vs. LoLM).** (1) Can the main computational strategy accomplish the task once its details are correct? If not, label GLM. (2) If so, and the error is confined to a condition, index, update, or initialization, label LoLM. The number of lines a fix needs plays no part; a single line can be GLM (Case S3.1).

- **Rule 5.4 (CD vs. DC).** A duplicated segment is labeled CD only, even if its result is unused. DC is added only for a separate, independent segment of unreachable or unused code (Case S3.5).

- **Rule 5.5 (NCC, SE, and II).** Output whose body is explanatory text is NCC, without SE; natural language mixed into code that makes it unparsable is also NCC. Code that itself violates the syntax rules is SE. A complete structure with only a `pass` or TODO placeholder is II (Case S3.6).

- **Rule 5.6 (RSI vs. RO).** RSI records a boundary case or exception admitted by the task that is left unhandled, or an unsafe use of inputs; RO records a demand for time, memory, or recursion depth that exceeds the limits. A failure caused by excessive demand is RO even when it surfaces as an exception such as `RecursionError` or `MemoryError` (Case S3.2). A wrong result under a boundary input is a computational error (GLM or LoLM by Rule 5.3); only a crash or unhandled exception under a boundary input is RSI.

- **Rule 5.7 (CI vs. LoLM).** An error that violates an explicit convention in the problem statement, the given code, or code the model itself already wrote is CI; a local computational error that violates no explicit convention is LoLM (Case S3.7).

- **Rule 5.8 (AKE vs. DSM).** Decide whose contract is violated: members and calling rules of an external library, a module, or the standard library give AKE; a data structure given by the task or built by the code gives DSM (Case S3.4).

S2.5 Generation Modes, I/O Adaptation, and Test Execution

Rule 6.1 (Generation mode). The generation mode sets the delivery requirement (Table S3; Case S3.3).

Table S3: Generation modes and delivery requirements (Rule 6.1)

Mode	Applicable tasks	Delivery requirement	Missing I/O handling
Function/method completion	HumanEval; APPS tasks that give a function signature	Implement the given signature without adding I/O code	Not labeled
Standalone script	APPS tasks without a function signature	Read from standard input and write to standard output	Labeled II

Rule 6.2 (I/O adaptation). If a sample implements the core function but lacks the required I/O handling, an external wrapper may connect it to the evaluation interface to test the core logic. The wrapper performs only the adaptations permitted in Table S4; raw and wrapper code are stored separately, and labels still apply to the raw output, so the missing I/O handling remains II.

Rule 6.3 (Test results). Test results are evidence, not a filter; every sample is annotated manually. Passing samples are still checked item by item; for failing samples, the subcategory follows from the failing cases, error messages, and minimal counterexamples. A test failure by itself maps to no subcategory.

Rule 6.4 (Execution environment). Code runs in isolation (no external network, unprivileged user, dedicated temporary directory). Time and memory limits follow the benchmark's

Table S4: Permitted scope of I/O adaptation (Rule 6.2)

Permitted interface adaptation	**Not permitted (code repair)**
Parse standard input into function arguments as the problem statement specifies	Modify algorithms, conditions, loop bounds, constants, or data structures
Call the entry function the model wrote	Add missing functionality, branches, or return values
Print the return value in the format the problem statement specifies	Mask type errors in the implementation with type conversions
Build a separate test driver for function-completion tasks	Catch exceptions and return default values or reference-solution results

official evaluation scripts; if neither the problem nor the benchmark sets them, protective defaults apply: 5 seconds per test case, 30 seconds cumulative per sample, and 512 MiB of memory. A sample exceeding only these defaults, with no corresponding limit in the problem, is recorded as "execution-limited" and not labeled directly; the annotator decides on RO by complexity analysis.

S2.6 Reuse Protocol

The protocol applies the thirteen subcategories to outputs of new models or model versions. It does not repeat open coding: annotation starts from the established definitions, rules, and cases, and whether the existing categories cover the new outputs is checked continuously.

S2.6.1 Parameters and Personnel

Table S5 lists the protocol parameters, derived from the data of the original study; reusers may scale them proportionally to their budget and must report any scaling. Table S6 lists the roles.

S2.6.2 Task Sampling and Batches

- **Rule 9.1 (Anchor tasks).** Use the 100 anchor tasks (Table S5) for comparability with existing models. Other task domains form a separate partition, reported separately and never merged with the anchor results.
- **Rule 9.2 (Generation).** Generate 5 outputs per task with the original prompt templates and generation modes; record the model identifier, version or snapshot, call date, decoding parameters, and maximum output length. Duplicate outputs are kept and not resampled.
- **Rule 9.3 (Batching).** Split the 100 tasks into 5 batches of 20 (about 7 HumanEval, 7 APPS introductory, and 6–7 APPS interview), keeping all 5 outputs of a task in one batch. Samples are shown in random order with the model name and generation index hidden; test results are shown as evidence.
- **Rule 9.4 (Extension batches).** Each extra batch draws 20 new tasks from the same three source strata, disjoint from the anchor and calibration tasks.

S2.6.3 Annotator Calibration

- **Rule 10.1 (Calibration set).** 60 samples from the released data, with no task shared with the anchor tasks: 20 instructional examples with gold labels and adjudication rationales, for study; 40 test examples, of which 30 contain hallucinations (each subcategory at least twice; at least 6 multi-label samples) and 10 are hallucination-free. Each of four boundary groups (GLM/LoLM, RSI/RO, AKE/DSM, and II under different generation modes) has at least 3 samples.

Table S5: Reuse-protocol parameters

Parameter	Value	Rationale
Anchor tasks	100 stratified tasks: 33 HumanEval, 33 APPS introductory, 34 APPS interview (sampling seed 20260916)	Same set as the temperature experiment; direct comparison with the six models and three temperature conditions
Generations per task	5	As in the original study; 10 within-task pairs per task
Initial sample	500 outputs (5 batches)	NCC (0.41%) occurs at least once with probability ≈0.87, every other subcategory >0.99; 95% CI half-width of the overall rate 4–7 percentage points, depending on within-task correlation
Batch	100 outputs (20 tasks × 5 generations); tasks stratified by source	About 27 person-hours; quality checked per batch
Personnel	At least 2 annotators and 1 adjudicator; 4 annotators in rotating pairs recommended	Agreement reflects the team, not one fixed pair; two raters per sample, hence Fleiss' κ
Calibration set	60 released CoDelusion samples (20 instructional, 40 test); no task overlap with the anchor tasks	Final CoDelusion labels as gold standard; each subcategory at least twice; the four confusable boundary groups covered
Calibration pass	Against gold labels: presence agreement ≥0.90; exact label-set agreement ≥0.70; mean Jaccard ≥0.80	Original overall κ 0.89; exact multi-label match is stricter than a binary judgment
Agreement thresholds	Checked for a subcategory after 20 cumulative positive decisions (labeled by at least one annotator): Fleiss' $\kappa \geq 0.70$ and positive agreement ≥0.70; overall presence $\kappa \geq 0.80$	κ unstable below 20 positive decisions; 0.70 is "substantial agreement"
Pending proportion	≤5%	Relabeling all pending samples shifts the overall rate by at most 5 points, about the CI half-width
Stopping condition	After the 5 anchor batches: no new-category candidate in the 2 most recent consecutive batches; agreement and pending criteria met	Saturation check: base sample, consecutive batches, new-information threshold
New category	At least 5 instances from at least 3 tasks that no existing subcategory expresses after adjudication	About 1% of 500 outputs; excludes task-specific sporadic cases
Extension cap	At most 5 extra batches (1,000 outputs in total), then taxonomy revision (Rule 13.3)	Bounds cost; explicit exit when the taxonomy no longer fits
Annotation speed	About 8.2 min per annotation pass	Original study: 20 people × 45 days × about 4 h ≈ 3,600 person-hours for 13,125 samples × 2 = 26,250 passes
Adjudication	Planned for 20% of samples, about 10 min each	Planning value; updated to the observed rate after the first batch
I/O adaptation	About 2.9% of samples, about 16 min each	Original study: 428 samples (2.85%), about 112 person-hours

Table S6: Roles, requirements, and responsibilities

Role	Requirements	Responsibilities
Annotator	At least 4 years of Python experience and familiarity with common algorithms (as for the annotators of the original study); passed calibration (Rule 10.3)	Independent annotation; evidence records; flagging unmapped phenomena
Adjudicator	At least 6 years of Python experience; took part in building or reviewing the calibration set	Resolving disagreements and "specification under verification" samples; recording rationales; confirming new-category candidates
Coordinator	May be the adjudicator	Assigning batches; computing agreement; guideline reviews; version management

- **Rule 10.2 (Procedure).** Study the guideline and instructional examples (about 2 hours), complete the 40 test examples independently (about 5.5 hours), and review errors case by case with the coordinator (about 1 hour).
- **Rule 10.3 (Pass criteria).** Against the gold labels, all of: presence agreement $\geq$0.90 (at least 36 of 40), exact label-set agreement $\geq$0.70 (at least 28), and mean Jaccard $\geq$0.80. An annotator who fails reviews the errors and takes 20 further test examples at the same thresholds; a second failure excludes the annotator from the current round.

S2.6.4 Independent Annotation and Adjudication

- **Rule 11.1 (Independent annotation).** Two annotators label each sample independently and see no other labels before submission. Each issue records the code location, violated requirement, evidence, and subcategory (Rule 2.1).
- **Rule 11.2 (Unmapped phenomena).** An issue no existing subcategory can express is recorded as "unmapped" with the phenomenon, code location, and rationale, optionally with the closest existing subcategory; the record enters the coverage check (Rule 12.3).
- **Rule 11.3 (Disagreements).** Any difference between the two label sets is a disagreement for the adjudicator, who reads both labels and evidence, adds counterexamples or execution checks if needed, sets the final labels, and records the source as category boundary, evidence interpretation, task specification, execution environment, or missed label.
- **Rule 11.4 (Pending samples).** If information needed for adjudication is unavailable (e.g., the problem statement conflicts with the tests, or the execution environment fails), the sample is marked "pending" with the reason. Pending samples are resolved or reported separately before the round ends and are never counted as empty label sets.

S2.6.5 Quality Control and Convergence

After each batch, the coordinator runs the checks of Rules 12.1–12.3 and enters the results in the batch record.

- **Rule 12.1 (Agreement check).** On the independent labels before adjudication, compute cumulatively each subcategory's Fleiss' κ and positive agreement and the overall κ for hallucination presence (Rule 12.4). A guideline review is triggered if a subcategory with at least 20 cumulative positive decisions has $\kappa < 0.70$ or positive agreement < 0.70, or if the overall $\kappa < 0.80$:
  1. the adjudicator re-examines all disagreements on that subcategory in the batch and classifies their sources (Rule 11.3);
  2. for the sources "category boundary" and "evidence interpretation", the rules are supplemented or clarified, with cases added if needed, the guideline version is raised to 1.x (Rule 13.1), and all annotators are briefed;
  3. affected samples in the batch are re-examined under the revised rules;
  4. the subcategory is checked again in the next batch; the review closes after two consecutive batches meet the thresholds.
- **Rule 12.2 (Pending check).** If cumulative pending samples exceed 5%, the next batch is suspended until the pending samples are resolved.
- **Rule 12.3 (Coverage check).** The adjudicator reviews every unmapped record: issues expressible by existing subcategories are relabeled, with clarifications where needed; the

others are registered as new-category candidates. A candidate with at least 5 instances from at least 3 tasks proceeds to Rule 13.2.

- **Rule 12.4 (Agreement computation).** For a subcategory, let a be the number of samples both annotators label with it, b and c the numbers labeled by only one annotator, d the number labeled by neither, and $n = a + b + c + d$. With two raters per sample, $p_o = (a + d)/n$, $\bar{p} = (2a + b + c)/(2n)$, $p_e = \bar{p}^2 + (1 - \bar{p})^2$, $\kappa = (p_o - p_e)/(1 - p_e)$, and positive agreement is $2a/(2a+b+c)$. Exact label-set agreement and the sample-level mean Jaccard are also reported (Jaccard is 1 when both annotators assign the empty set).

- **Rule 12.5 (Stopping condition).** After the 5 anchor batches, the round ends when all of the following hold:

  1. no further new-category candidate in the 2 most recent consecutive batches;
  2. cumulative pending proportion $\leq$5%;
  3. every subcategory with at least 20 cumulative positive decisions, and the overall κ, meet the thresholds of Rule 12.1.

  Otherwise one batch is added (Rule 9.4) and the checks are repeated; if they still fail after 5 extra batches (1,000 outputs in total), taxonomy revision follows (Rule 13.3). Subcategories with fewer than 20 cumulative positive decisions are excluded from the threshold assessment; the report lists their counts and all their disagreement cases.

S2.6.6 New Categories and Versions

- **Rule 13.1 (Minor version, 1.x).** Clarifies definition wording or adds boundary rules or cases without changing the meaning of any subcategory. Only affected samples in the current batch are re-examined; labels of completed batches stay unchanged.

- **Rule 13.2 (New category).** A candidate is confirmed only with at least 5 instances from at least 3 tasks and a joint finding by the adjudicator and coordinator that clarifying existing boundaries cannot express it. A new major version (2.0) then defines the subcategory and its boundaries with the existing ones, and all completed batches are searched for possible instances, which are re-examined.

- **Rule 13.3 (Taxonomy revision).** If the stopping condition is not met within the extension cap, the unmapped and high-disagreement samples are re-coded with the open-coding method of the original study to form a new major version; the report states the reasons for and scope of the revision.

- **Rule 13.4 (Version records).** Each label records the guideline version used. New versions never overwrite old labels; comparisons report statistics under both versions.

S2.6.7 Workload, Reporting, and Record Format

Table S7 gives the workload for 500 outputs with the minimum configuration (2 annotators, 1 adjudicator); annotation and adjudication take about 154 person-hours. With 4 annotators working about 4 hours per day each, independent annotation takes about 9 working days and one extension round, including calibration and adjudication, about 3 weeks. Each additional batch adds about 33 person-hours (27 annotation, 3.3 adjudication, 0.8 I/O adaptation, 2 quality checks).

A report on an extension round includes at least:

1. model identifier, version or snapshot, call date, decoding parameters, and prompt-template version;

Table S7: Workload for 500 outputs with the minimum configuration

Stage	Calculation	Person-hours
Calibration	3 people × about 8.5 h (study, test, review)	26
Independent annotation	500 × 2 passes × 8.2 min	137
Adjudication	500 × 20% × 10 min	17
I/O adaptation	500 × 2.9% × 16 min	4
Quality checks and coordination	5 batches × about 2 h	10
Total		about 194

2. task list, batch partition, and number of samples completed;
3. overall and per-subcategory occurrence rates with task-level bootstrap 95% confidence intervals;
4. comparison with existing models on the anchor tasks;
5. per-subcategory positive decisions, Fleiss' κ, and positive agreement; overall κ; exact label-set agreement;
6. distribution of disagreement sources, guideline-review records, and number of pending samples with reasons;
7. unmapped records, new-category candidates, and their outcomes;
8. guideline version and person-hours spent.

Each sample is stored as one record with the fields below; each entry of `issues` holds the three items of Rule 2.1. Labels use the numeric codes 1.1–7.2 of the `hallucination` field in the released data; annotators appear under anonymous IDs.

```
{
  "sample_id": "APPS/train-6/NewModel/3",
  "task_id": "APPS/train-6",
  "model": {"name": "", "version": "", "call_date": "",
            "temperature": 0.8, "max_tokens": 0},
  "generation_mode": "function_completion | script_generation",
  "guideline_version": "1.0",
  "raw_output": "",
  "extracted_code": "",
  "execution": {"raw_status": "", "adapted_status": "",
                "io_adapter_path": null, "log": ""},
  "annotations": [
    {"annotator": "A1", "labels": ["2.1", "3.1"],
     "issues": [{"label": "2.1", "lines": [2, 3],
                 "violated_requirement": "", "evidence": ""}],
     "unmapped": []},
    {"annotator": "A3", "labels": ["2.1"], "issues": [], "unmapped": []}
  ],
  "adjudication": {"adjudicator": "J1", "disagreement_type": "missed label",
                   "rationale": "", "final_labels": ["2.1", "3.1"]},
  "status": "final | pending",
  "pending_reason": null
}
```

S2.7 Recording Multi-turn and Agentic Generation

In multi-turn or agentic generation, the code version produced in each turn is one annotation unit, judged on the task requirements, context, and feedback visible in that turn. The sample

record gains the fields in Table S8, from which per-subcategory fix, introduction, and final residual rates are computed. For agents that modify existing repositories, tool calls are also recorded to separate defects already present in the code from issues introduced in the current turn.

Table S8: Additional record fields for multi-turn and agentic generation

Field	Content
`turn`	Turn number
`feedback`	Feedback the model received before this turn (test results, error messages, user comments, tool outputs)
`diff`	Code difference from the previous version
`issue_transitions`	Status of each issue: newly introduced, persisting, fixed, or reintroduced

S2.8 Inter-annotator Agreement in CoDelusion

Table S9 reports the per-subcategory agreement of CoDelusion on the independent labels before adjudication.

Table S9: Inter-annotator Agreement per Subcategory

Subcategory	Positive Decisions	Fleiss' κ	Positive Agreement
DTM	350	0.792	0.794
DSM	513	0.816	0.819
GLM	6,894	0.872	0.901
CI	941	0.828	0.833
LoLM	4,200	0.846	0.868
II	856	0.803	0.808
CD	1,484	0.905	0.910
DC	450	0.851	0.853
AKE	542	0.884	0.886
RSI	1,116	0.762	0.771
RO	729	<u>0.717</u>	<u>0.724</u>
SE	638	**0.958**	**0.959**
NCC	122	0.934	0.934

Note: Positive Decisions counts both annotators' positive labels across 15,000 samples. Agreement formulas are given in Rule 12.4.

S3 Adjudication Cases

Each case gives the task requirement, the code, the competing judgments, the decision, and the rule applied. The cases cover all boundary rules of Section S2.4 and serve as instructional examples in calibration (Rule 10.1).

S3.1 GLM vs. LoLM: Two Errors on One Task

Task. Given a non-empty list of integers, return the maximum sum over its non-empty contiguous subarrays. Code A:

```
def max_subarray_sum(xs):
    return sum(xs)
```

- Candidate 1, LoLM: only one line is wrong, and replacing it suffices.
- Candidate 2, GLM: the code computes the sum of the whole list, not the maximum subarray sum.

Decision: GLM (Rule 5.3). For `[-2, 3, -1]` the code returns 0; the correct result is 3. The code computes a different objective, an error of the main strategy; the size of the fix is irrelevant.

Code B for the same task:

```
def max_subarray_sum(xs):
    best = 0
    current = 0
    for x in xs:
        current = max(x, current + x)
        best = max(best, current)
    return best
```

- Candidate 1, GLM: for an all-negative list the code returns 0, the sum of a non-existent subarray.
- Candidate 2, LoLM: the recurrence is correct; the error stems from the initialization `best = 0`.

Decision: LoLM (Rules 5.3 and 5.6). For `[-3, -1]` the code returns 0; the correct result is -1. The recurrence can accomplish the task and the error is confined to the initialization. An all-negative list is a permitted input, but the wrong result is a computational error, not RSI.

S3.2 RSI vs. RO: Unhandled Boundary vs. Resource-Limit Violation

Task. `sum_to(n)` returns $1 + 2 + \cdots + n$ for an integer $0 \leq n \leq 10^5$. Code:

```
def sum_to(n):
    if n == 0:
        return 0
    return n + sum_to(n - 1)
```

- Candidate 1, RSI: for $n = 10^5$ the code raises an uncaught `RecursionError`, a crash under a permitted input.
- Candidate 2, RO: the recursion depth that the algorithm requires grows linearly with n and exceeds the interpreter's limit.

Decision: RO (Rule 5.6). The recursion needs n nested calls, so for $n = 10^5$ its demand exceeds Python's default recursion limit of 1,000 frames. The exception is a consequence of this demand (Rule 2.3), and no boundary case is left unhandled, so RSI is not labeled.

Contrast: RSI only. A task requires `mean(xs)` to return the arithmetic mean of `xs` and 0 for an empty list. The code returns `sum(xs) / len(xs)`, which raises `ZeroDivisionError` for the permitted empty input. The empty list is a boundary case that the task admits and the code leaves unhandled, while the resource demand is within limits, so only RSI is labeled.

S3.3 II Depends on the Generation Mode

Task (standalone-script mode). Read two integers from one line of standard input and write their sum to standard output. Code:

```
def add(a, b):
    return a + b
```

- Candidate 1, empty label set: `add(2, 3)` returns the correct result.
- Candidate 2, II: the program neither reads input nor writes a result.

Decision: II (Rules 6.1 and 6.2). Standalone-script mode requires complete I/O handling, which the raw output lacks. The core logic may be tested with the external wrapper below; the tests then pass, but II is not withdrawn:

```
# External wrapper for evaluation; not part of the model output
a, b = map(int, input().split())
print(add(a, b))
```

Contrast. In function-completion mode ("Implement `add(a, b)`"), the same code receives an empty label set, since that mode requires that no I/O code be added.

S3.4 AKE vs. DSM: Whose Contract Is Violated?

Task. Using only the Python standard library or a self-written implementation, decide whether an integer n with $2 \leq n \leq 100$ is prime. Code:

```
import math

def is_prime(n):
    return math.isprime(n)
```

- Candidate 1, DSM: the code accesses a member that does not exist on an object.
- Candidate 2, AKE: the standard-library `math` module has no `isprime` function.

Decision: AKE (Rules 5.8 and 2.3). The violated contract is an interface fact of an external module, verifiable in the official Python documentation. The exception raised by the failed call is a consequence of the same defect and receives no further label.

Contrast: DSM. The task states that `record` is a dictionary with only the keys `"name"` and `"age"`, and the code accesses `record["username"]`. The violated contract is a data structure given by the task, so DSM is labeled.

S3.5 CD vs. DC: Duplicated vs. Ineffective Code

Task. Given a list of integers, return a new list of all its positive numbers multiplied by 2. Code:

```
def process_data(data):
    result = []
    for item in data:
        if item > 0:
            result.append(item * 2)

    processed = []
    for item in data:
        if item > 0:
            processed.append(item * 2)

    rate = len(result) / len(data) if data else 0
    return result
```

- Candidate 1, CD + DC + DC: lines 7–10 are a repeated computation whose result is unused, and `rate` on line 12 is also unused.
- Candidate 2, CD + DC.

Decision: CD + DC (Rules 5.4 and 2.2). Lines 7–10 repeat the computation of lines 2–5 and are labeled CD; their unused result belongs to the same duplication, so they receive no DC. Line 12 is a separate computation whose result is never used and is labeled DC. DC is recorded once per sample.

S3.6 NCC, SE, and II: Output Form

Task. Implement `sort_list(items)`, which returns a new sorted list. For each of the four outputs below, the competing judgments are NCC, SE, and II. Output A:

```
1 def sort_list(items):
2     """
3     To sort a list in Python, we would typically use the built-in sorted() function,
4     which returns a new sorted list.
5     """
```

Decision for A: NCC (Rule 5.5). The function body contains only explanatory text in place of an implementation. The code parses, so SE is not labeled.

Output B:

```
1 def sort_list(items):
2     # TODO: implement sorting
3     pass
```

Decision for B: II (Rule 5.5). The structure is complete with a `pass` placeholder; this is missing functionality, not NCC.

Output C:

```
1 def sort_list(items)
2     return sorted(items)
```

Decision for C: SE (Rule 5.5). The function definition lacks a colon; the code itself violates the syntax.

Output D:

```
1 def sort_list(items):
2     first we copy the list and then sort it
3     return sorted(items)
```

Decision for D: NCC (Rules 5.5 and 2.3). Natural language mixed into the code makes it unparsable; the parse failure is a consequence of the same defect, so SE is not added.

S3.7 CI vs. LoLM: Is Explicit Context Violated?

Task. Implement a stack class in which `push` pushes an element and `pop` pops and returns the most recently pushed element. Code:

```
1 class Stack:
2     def __init__(self):
3         self.items = []
4 
5     def push(self, item):
6         self.items.append(item)
7 
8     def pop(self):
9         return self.items.pop(0)
```

- Candidate 1, LoLM: only the index in `pop` is wrong.
- Candidate 2, CI: the problem statement explicitly requires last-in, first-out behavior, whereas the code implements first-in, first-out.

Decision: CI (Rule 5.7). The error directly violates the explicit conventions "stack" and "pop the most recently pushed element" in the problem statement.

Contrast 1: CI. Contradicting the model's own earlier code is also CI. Below, `is_valid` defined on line 2 takes three arguments, but line 7 passes two; this contradicts the model's own definition and is labeled CI, not DTM.

```
1 def count_valid(grid):
2     def is_valid(grid, r, c):
3         return grid[r][c] == 1
4     total = 0
5     for r in range(len(grid)):
6         for c in range(len(grid[0])):
7             if is_valid(r, c):
8                 total += 1
9     return total
```

Contrast 2: LoLM. A task requires counting the even numbers from 1 to n inclusive, but the code uses `range(1, n)`. No explicit convention in the problem statement is violated; this is a local loop-bound error, labeled LoLM.

S4 Temperature-Control Experiment

We sampled 100 of the 500 tasks stratified by source (33 HumanEval, 33 APPS introductory, and 34 APPS interview tasks). Each of the six models generated five outputs per task at T=0 and T=0.5 with the prompts in Section S1, and these outputs were compared with the existing T=0.8 outputs for the same tasks. Each model contributes 500 samples per temperature, giving 3,000 samples per temperature and 9,000 in total.

A sample is hallucinated if it carries any label. Per-category rates use all outputs as the denominator, and the logic-category rate counts outputs with at least one of GLM, CI, LoLM, or II. Temperature differences are paired differences over tasks after averaging the five outputs within each task, with the six models weighted equally. Confidence intervals come from a task-cluster bootstrap stratified by source (5,000 resamples) that keeps all models, temperatures, and generations of each sampled task. Tables S10 and S11 report the results.

Table S10: Overall Hallucination Prevalence at Three Temperatures

Model	**T=0,** n **(%)**	**T=0.5,** n **(%)**	**T=0.8,** n **(%)**	**Δ(0.8−0), pp**	**95% CI**
CodeLlama-7B-I	345 (69.0)	343 (68.6)	324 (64.8)	−4.20	$[-11.20, 3.00]$
DSC-V2-Lite-I-16B	222 (44.4)	218 (43.6)	253 (50.6)	6.20	$[-2.60, 15.00]$
Qwen2.5-Coder-32B-I	208 (41.6)	212 (42.4)	248 (49.6)	8.00	$[-0.40, 16.40]$
GPT-4o	160 (32.0)	159 (31.8)	199 (39.8)	7.80	$[1.40, 14.20]$
DeepSeek-V2.5-1210	135 (27.0)	138 (27.6)	165 (33.0)	6.00	$[-2.40, 14.40]$
Claude-3.5-Sonnet	140 (28.0)	150 (30.0)	208 (41.6)	13.60	$[6.20, 21.20]$
All six models	1,210 (40.33)	1,220 (40.67)	1,397 (46.57)	6.23	$[2.37, 10.17]$

Note: pp = percentage points. All five hallucination judgments agree in 85.67%, 75.50%, and 72.17% of task–model pairs at T=0, 0.5, and 0.8, respectively.

Table S11: Subcategory Prevalence at Three Temperatures (n = 3,000 per temperature)

Subcategory	**T=0**	**T=0.5**	**T=0.8**
DTM	27 (0.90%)	25 (0.83%)	35 (1.17%)
DSM	45 (1.50%)	51 (1.70%)	54 (1.80%)
GLM	618 (20.60%)	593 (19.77%)	741 (24.70%)
CI	45 (1.50%)	53 (1.77%)	82 (2.73%)
LoLM	428 (14.27%)	442 (14.73%)	426 (14.20%)
II	189 (6.30%)	196 (6.53%)	62 (2.07%)
CD	0 (0.00%)	1 (0.03%)	141 (4.70%)
DC	17 (0.57%)	10 (0.33%)	50 (1.67%)
AKE	19 (0.63%)	10 (0.33%)	43 (1.43%)
RSI	22 (0.73%)	14 (0.47%)	125 (4.17%)
RO	162 (5.40%)	165 (5.50%)	59 (1.97%)
SE	17 (0.57%)	24 (0.80%)	84 (2.80%)
NCC	31 (1.03%)	28 (0.93%)	15 (0.50%)

Note: Labels overlap. Outputs with any logic label number 1,083, 1,089, and 1,243 (89.50%, 89.26%, and 88.98% of hallucinated outputs) at T=0, 0.5, and 0.8, respectively.

S5 Type Classification: Evaluation Rules and Per-category Results

Each hallucinated test sample receives one predicted subcategory, and seven-category predictions are obtained by mapping it to its top-level category. A prediction is correct if it belongs to the sample's label set. For per-category metrics, each sample has one reference category: the predicted category if the prediction is correct, and otherwise the lowest-numbered category in its label set. NONE or unparsable outputs count as errors and as predictions of no category. Precision, recall, and F1 are computed one-vs-rest against the reference categories; macro-F1 averages F1 over categories, and weighted metrics weight each category by its number of reference samples, so that accuracy equals weighted recall. Tables S12 and S13 give the per-category results.

Table S12: Per-category Precision, Recall, and F1 for the Seven Top-level Categories

	DeepSeek-V3			**Qwen3**			**GPT-4o**		
Category	P	R	F1	P	R	F1	P	R	F1
DSI	0.323	0.588	0.417	0.139	0.647	0.229	0.072	0.647	0.129
LI	0.915	0.923	0.919	0.860	0.701	0.772	0.926	0.638	0.755
RDC	0.700	0.560	0.622	0.600	0.480	0.533	0.588	0.400	0.476
EKI	0.895	0.971	0.932	0.927	0.729	0.816	0.963	0.371	0.536
RSI	0.000	0.000	0.000	0.083	0.143	0.105	0.000	0.000	0.000
RMI	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
SSI	0.900	0.375	0.529	0.688	0.458	0.550	0.625	0.417	0.500
Macro-F1			0.488			0.429			0.342

	Claude-3.5			**Llama-3.1**		
Category	P	R	F1	P	R	F1
DSI	0.071	0.647	0.129	0.049	0.529	0.090
LI	0.909	0.570	0.701	0.802	0.473	0.595
RDC	0.647	0.440	0.524	0.389	0.280	0.326
EKI	0.949	0.529	0.679	0.784	0.414	0.542
RSI	0.111	0.143	0.125	0.000	0.000	0.000
RMI	0.000	0.000	0.000	0.000	0.000	0.000
SSI	0.733	0.458	0.564	0.444	0.333	0.381
Macro-F1			0.389			0.276

Table S13: Per-category Precision, Recall, and F1 for the Thirteen Subcategories

Subcategory	Metric	DeepSeek-V3	Qwen3	GPT-4o	Claude-3.5	Llama-3.1
	P	0.300	0.091	0.048	0.058	0.038
DTM	R	0.429	0.429	0.429	0.429	0.286
	F1	0.353	0.150	0.086	0.102	0.068
	P	0.333	0.128	0.070	0.072	0.049
DSM	R	0.500	0.500	0.500	0.500	0.400
	F1	0.400	0.204	0.123	0.127	0.087
	P	0.709	0.571	0.644	0.690	0.533
GLM	R	0.732	0.442	0.342	0.411	0.253
	F1	0.720	0.499	0.447	0.515	0.343
	P	0.538	0.323	0.375	0.435	0.240
CI	R	0.467	0.333	0.300	0.333	0.200
	F1	0.500	0.328	0.333	0.377	0.218
	P	0.612	0.415	0.491	0.540	0.370
LoLM	R	0.571	0.371	0.267	0.324	0.190
	F1	0.591	0.392	0.346	0.405	0.252
	P	0.583	0.370	0.421	0.474	0.286
II	R	0.538	0.385	0.308	0.346	0.231
	F1	0.560	0.377	0.356	0.400	0.255
	P	0.625	0.412	0.462	0.538	0.286
CD	R	0.526	0.368	0.316	0.368	0.211
	F1	0.571	0.389	0.375	0.438	0.242
	P	0.400	0.200	0.200	0.250	0.100
DC	R	0.333	0.167	0.167	0.167	0.167
	F1	0.364	0.182	0.182	0.200	0.125
	P	0.838	0.791	0.870	0.879	0.667
AKE	R	0.886	0.486	0.286	0.414	0.257
	F1	0.861	0.602	0.430	0.563	0.371
	P	0.000	0.067	0.000	0.100	0.000
RSI	R	0.000	0.143	0.000	0.143	0.000
	F1	0.000	0.091	0.000	0.118	0.000
	P	0.000	0.000	0.000	0.000	0.000
RO	R	0.000	0.000	0.000	0.000	0.000
	F1	0.000	0.000	0.000	0.000	0.000
	P	0.778	0.583	0.500	0.636	0.385
SE	R	0.368	0.368	0.316	0.368	0.263
	F1	0.500	0.452	0.387	0.467	0.312
	P	0.500	0.333	0.333	0.333	0.200
NCC	R	0.400	0.200	0.200	0.200	0.200
	F1	0.444	0.250	0.250	0.250	0.200
Macro-F1		0.451	0.301	0.255	0.305	0.190

Note: P = Precision; R = Recall.